

net-01 - Introdução ao Open vSwitch(OVS)

Contexto [↗](#)

Neste treinamento iremos abordar:

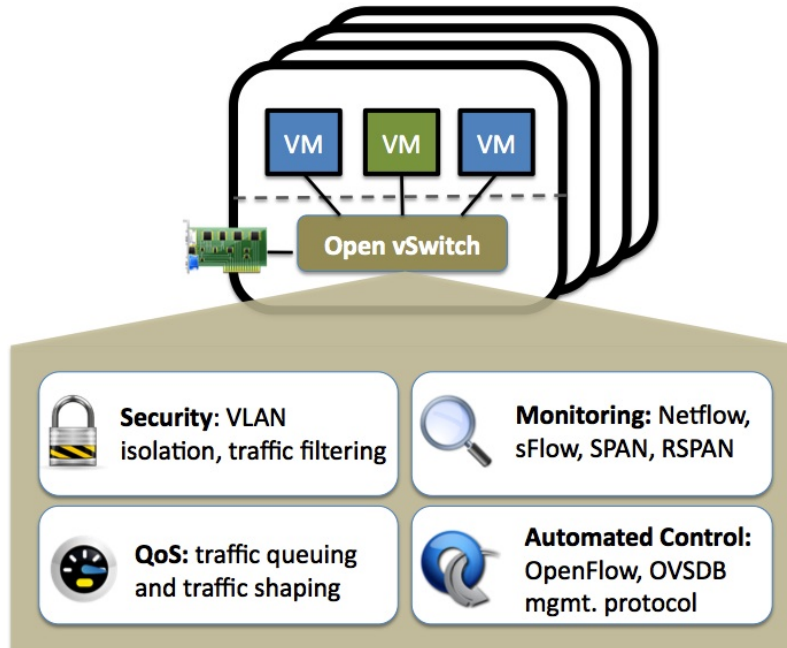
- Uma breve introdução ao Open vSwitch
- Hands on - instalação do Open vSwitch e criação de uma bridge
- Hands on - criando conectividade entre dois namespaces via OVS
- Componentes Core do Open vSwitch

Introdução ao Open vSwitch [↗](#)

OVS é a abreviação para **Open vSwitch**, que é um projeto open source utilizado para **interconectar** VMs/containers via L2 ou L3 em um mesmo host ou entre diferentes hosts.

É uma solução muito popular utilizada em deployments com Openstack e hypervisor KVM e o OVS suporta muitas features convencionais de [switches](#) como:

- VLAN tagging
- LACP
- STP
- QoS
- Tunneling protocols (GRE, VXLAN)
- ACLs
- etc...



As últimas versões do OVS podem ser encontrados no site oficial [Open vSwitch](#) e que também é distribuído para vários sistemas operacionais além das distribuições Linux.

Instalação do Open vSwitch e criação de uma bridge [↗](#)



Para fins de teste **apenas**, se recomenda ter uma VM(sandbox) executando o Ubuntu 20.04 com os pacotes latest instalados para seguir os exercícios.

1 - Instalando o OVS via pacote [↗](#)

Existem várias versões do OVS suportadas para o Ubuntu 20.04 e que pode ser instalado a versão conforme a necessidade.

Por padrão no Ubuntu 20.04 o OVS instalado será o 2.13, porém é possível instalar os pacotes para as últimas versões 2.16 e 2.17 habilitando um dos dois repositórios a seguir.

Para OVS 2.16:

```
1 # add-apt-repository -y cloud-archive:xena
```

Para OVS 2.17:

```
1 # add-apt-repository -y cloud-archive:yoga
```

Exemplo de como desabilitar um dos repositórios acima e manter o padrão:

```
1 # add-apt-repository -y -r cloud-archive:xena
```

Para realizar a instalação seguir:

```
1 # apt install openvswitch-switch
```

Checar a versão do pacote do OVS instalado:

```
1 # dpkg -l | grep openvswitch-switch
2 ii  openvswitch-switch      2.17.0-0ubuntu1-cloud0      amd64      Open vSwitch switch implementa
3
```

Após a instalação ser concluída, verificar se o serviço está em execução:

```
1 # systemctl status openvswitch-switch
```

Iremos criar uma bridge para começar a testar as funcionalidade do OVS. Essa bridge será utilizada no próximo exercício e é necessário que esse passo seja realizado.

No momento, o OVS não possui nenhuma bridge ou portas configuradas conforme abaixo:

```
1 # ovs-vsctl show
2 eb749a0c-06f6-475b-b5cf-4969cd9c7ba3
3     ovs_version: "2.17.0"
```

Criando a bridge "mybridge":

```
1 # ovs-vsctl add-br mybridge
```

Após criar é possível checar que a mesma foi adicionada:

```
1 # ovs-vsctl show
2 eb749a0c-06f6-475b-b5cf-4969cd9c7ba3
3     Bridge mybridge
4         Port mybridge
5             Interface mybridge
6                 type: internal
7     ovs_version: "2.17.0"
```

Nesse momento a bridge mybridge não possui nenhuma porta além de ela mesma porém por padrão ela estará down no sistema operacional.

Você pode validar com o seguinte comando:

```
1 # ip link show mybridge
2 5: mybridge: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN mode DEFAULT group default qlen 1000
3    link/ether c6:c8:f9:23:a5:47 brd ff:ff:ff:ff:ff:ff
```

Será necessário habilitar essa bridge conforme abaixo:

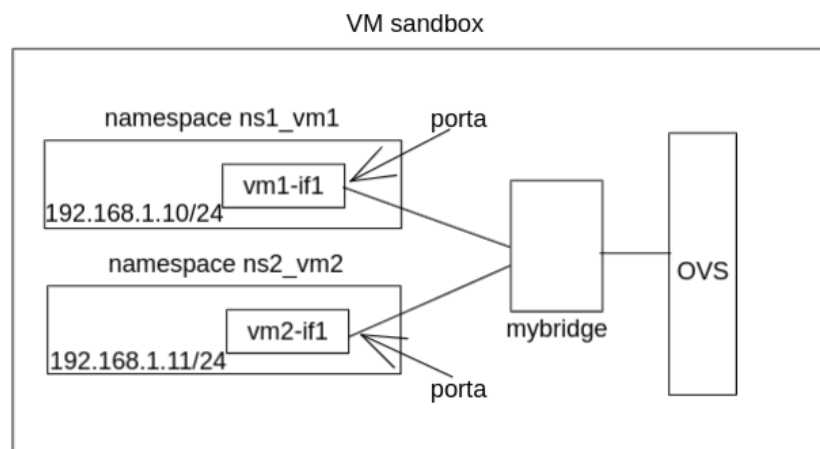
```
1 # ip link set dev mybridge up
2 # ip link show mybridge
3 5: mybridge: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN mode DEFAULT group default ql
4    link/ether c6:c8:f9:23:a5:47 brd ff:ff:ff:ff:ff:ff
```

Um dos arquivos de log do OVS é o **/var/log/openvswitch/ovs-vswitchd.log**, que geralmente irá logar ações realizadas no OVS e como erros:

```
1 # tail -f /var/log/openvswitch/ovs-vswitchd.log
2 2022-05-27T17:31:10.171Z|00034|bridge|INFO|bridge mybridge: added interface mybridge on port 65534
3 2022-05-27T17:31:10.171Z|00035|bridge|INFO|bridge mybridge: using datapath ID 0000c6c8f923a547
4 2022-05-27T17:31:10.171Z|00036|netdev_linux|WARN|query mybridge qdisc failed (Operation not supported)
5 2022-05-27T17:31:10.171Z|00037|netdev_linux|WARN|mybridge: removing policing failed: Operation not supported
6 2022-05-27T17:31:10.171Z|00038|connmgr|INFO|mybridge: added service controller "punix:/var/run/openvswitch/mybri
7 2022-05-27T17:31:10.173Z|00039|netdev_linux|WARN|mybridge: removing policing failed: Operation not supported
8 2022-05-27T17:31:20.174Z|00040|memory|INFO|peak resident set size grew 309% in last 10.0 seconds, from 13308 kB
9 2022-05-27T17:31:20.174Z|00041|memory|INFO|handlers:2 idl-cells:96 ports:1 revalidators:2 rules:5
10 2022-05-27T17:36:44.945Z|00042|netdev_linux|WARN|mybridge: removing policing failed: Operation not supported
11 2022-05-27T17:36:44.948Z|00043|netdev_linux|WARN|mybridge: removing policing failed: Operation not supported
12
```

2 - Criando conectividade entre dois namespaces via OVS [↗](#)

Para esse exercício, iremos estudar o seguinte diagrama e iniciar as configurações:



No diagrama acima, temos representado da direita para a esquerda o OVS gerenciando a bridge **mybridge** e a mesma conectando 2 namespaces. O uso de namespace nesse exercício é para simular uma VM ou container, já que o mesmo é uma partição virtual do Kernel Linux que irá ter uma stack de rede (nic e IP address) própria onde vamos conectar uma porta conectada na bridge **mybridge**.

[i](#) Mais informações sobre [namespaces do Linux](#).

Nesse cenário vamos criar a conectividade L2 entre os namespaces e dessa forma será possível ter a conectividade L3 entre eles.

Após entender o diagrama, podemos iniciar a criação dos 2 namespaces:

```
1 # ip netns add ns1_vm1
2 # ip netns add ns2_vm2
```

Verifique os namespaces criados com o seguinte comando:

```
1 # ip netns
2 ns2_vm2
3 ns1_vm1
```

Agora serão criadas as portas na bridge **mybridge** dos respectivos namespaces:

```
1 # ovs-vsctl add-port mybridge vm1-if1 -- set interface vm1-if1 type=internal
2 # ovs-vsctl add-port mybridge vm2-if1 -- set interface vm2-if1 type=internal
```

Para checar as portas criadas na bridge **mybridge**:

```
1 # ovs-vsctl show
2 eb749a0c-06f6-475b-b5cf-4969cdb7ba3
3     Bridge mybridge
4         Port vm1-if1
5             Interface vm1-if1
6                 type: internal
7         Port vm2-if1
8             Interface vm2-if1
9                 type: internal
10        Port mybridge
11            Interface mybridge
12                type: internal
13    ovs_version: "2.17.0"
```

Nesse momento estamos prontos para conectar as portas criadas nos namespaces:

```
1 # ip link set vm1-if1 netns ns1_vm1
2 # ip link set vm2-if1 netns ns2_vm2
```

Após conectar as portas nos namespaces é possível verificar que as portas estão presentes porém desabilitadas:

```
1 # ip netns exec ns1_vm1 ip a
2 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4 6: vm1-if1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
5     link/ether 7a:2e:15:c2:93:37 brd ff:ff:ff:ff:ff:ff
6 # ip netns exec ns2_vm2 ip a
7 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
8     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
9 7: vm2-if1: <BROADCAST,MULTICAST> mtu 1500 qdisc noop state DOWN group default qlen 1000
10    link/ether b6:93:74:80:62:23 brd ff:ff:ff:ff:ff:ff
```

O próximo passo será habilitar as portas e realizar a configuração do IP conforme o diagrama:

```
1 # ip netns exec ns1_vm1 ip link set vm1-if1 up
2 # ip netns exec ns2_vm2 ip link set vm2-if1 up
3 # ip netns exec ns1_vm1 ip addr add 192.168.1.10/24 dev vm1-if1
4 # ip netns exec ns2_vm2 ip addr add 192.168.1.11/24 dev vm2-if1
```

Agora já é possível ver o status das portas e seu respectivo endereço IP:

```

1 # ip netns exec ns1_vm1 ip a
2 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
3     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
4 6: vm1-if1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
5     link/ether 7a:2e:15:c2:93:37 brd ff:ff:ff:ff:ff:ff
6     inet 192.168.1.10/24 scope global vm1-if1
7         valid_lft forever preferred_lft forever
8     inet6 fe80::782e:15ff:fec2:9337/64 scope link
9         valid_lft forever preferred_lft forever
10
11 # ip netns exec ns2_vm2 ip a
12 1: lo: <LOOPBACK> mtu 65536 qdisc noop state DOWN group default qlen 1000
13     link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
14 7: vm2-if1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc noqueue state UNKNOWN group default qlen 1000
15     link/ether b6:93:74:80:62:23 brd ff:ff:ff:ff:ff:ff
16     inet 192.168.1.11/24 scope global vm2-if1
17         valid_lft forever preferred_lft forever
18     inet6 fe80::b493:74ff:fe80:6223/64 scope link
19         valid_lft forever preferred_lft forever

```

Após checar que as portas possuem seus respectivos endereços IPs, faça um simples teste de conectividade via icmp da VM1 para VM2 conforme abaixo:

```

1 # ip netns exec ns1_vm1 ping -c 4 192.168.1.11
2 PING 192.168.1.11 (192.168.1.11) 56(84) bytes of data.
3 64 bytes from 192.168.1.11: icmp_seq=1 ttl=64 time=0.211 ms
4 64 bytes from 192.168.1.11: icmp_seq=2 ttl=64 time=0.066 ms
5 64 bytes from 192.168.1.11: icmp_seq=3 ttl=64 time=0.063 ms
6 64 bytes from 192.168.1.11: icmp_seq=4 ttl=64 time=0.066 ms
7
8 --- 192.168.1.11 ping statistics ---
9 4 packets transmitted, 4 received, 0% packet loss, time 3055ms
10 rtt min/avg/max/mdev = 0.063/0.101/0.211/0.063 ms

```

É possível checar os mac addresses aprendidos pelo OVS na bridge **mybridge**:

```

1 # ovs-appctl fdb/show mybridge
2 port  VLAN  MAC  Age
3     1     0  7a:2e:15:c2:93:37  54
4     2     0  b6:93:74:80:62:23  54

```

Até o momento o que foi criado é simplesmente uma comunicação leste-oeste no mesmo L2 conforme uma VLAN funcionaria em um switch convencional.

Isso é a coisa mais simples que o OVS pode realizar e conforme o número de workloads(VMs/containers) e Hypervisors(KVM) aumentam existe a necessidade de automação e a configuração automática do OVS acaba sendo um pré-requisito, vamos falar em outro treinamento como o Openflow pode programar o OVS e como ele funciona.

No momento, para que a comunicação entre os 2 namespaces seja possível foi instalado automaticamente um flow via Openflow para permitir o tráfego de rede:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=8791.539s, table=0, n_packets=95, n_bytes=7666, priority=0 actions=NORMAL
3

```

Um flow é um fluxo de comunicação de rede, que em nosso caso o fluxo acima tem a action de **NORMAL** que quer dizer que é permitida.

Componentes Core do Open vSwitch [↗](#)

Os 3 principais componentes do OVS são:

- **ovsdb-server** - gerencia uma base de dados local que armazena as configurações que serão aplicadas no ovs-vswitchd. Essa base de dados é persistente, portanto se o host reiniciar os dados estarão em disco.
 - Os tools que gerenciam essa base de dados são: **ovs-vsctl**, **ovsdb-client**
- **ovs-vswitchd** - executa em userspace e é responsável em processar todos flows Openflow e repassar para o Kernel.
 - Para interagir diretamente com ele use esse tool: **ovs-appctl**, **ovs-ofctl**
- **Kernel Datapath module(Datapath)** - responsável em realizar o forward dos pacotes e possui uma grande performance por executar o processamento dos flows(**Physical flows**) diretamente no nível do Kernel.
 - Tool de interação utilizado: **ovs-dpctl**

Basicamente uma Controller externa é responsável em programar como o Datapath irá manipular diferentes tipos de pacotes no formato de **flows**, também conhecidos como Physical flows.

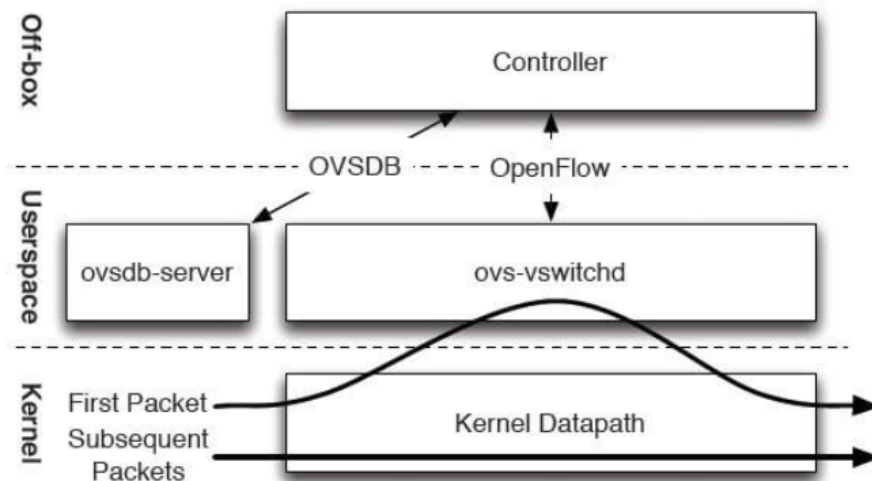
Um flow descreve como o Datapath deve manipular os pacotes de um tipo no formato de **actions**. Tipos de **action** incluem encaminhar o pacote para outra porta, output, modify vlan tag e etc.

O processo de encontrar um flow para um pacote é chamado de **flow matching**.

Considerando performance, parte dos flows são cached no Datapath(**fast path**) e outros no ovs-vswitchd (**slow path**).

Verificando o diagrama abaixo, podemos navegar pelo caminho tomado dos pacotes:

- um pacote entra no Datapath após ser recebido pela NIC do host
- se um flow é matched no Datapath para um pacote, o Datapath simplesmente irá executar a action definida
- caso contrário (**flow missing**), o Datapath irá enviar o pacote para o ovs-vswitchd (slow path) e outro processo de **flow matching** será realizado. Após o ovs-vswitchd determinar como o pacote deveria ser manipulado, o mesmo envia o pacote de volta para o Datapath com a action já determinada e também solicita ao Datapath para fazer o cache do flow para manipular posteriores pacotes.



Na database local do **ovsdb** são armazenadas várias informações do OVS em formato de tabelas. Para checar todas as tabelas dessa database execute o comando abaixo:

```
1 # ovsdb-client list-tables
2 Table
3 -----
4 Controller
5 Bridge
6 Queue
7 IPFIX
```

```

8 NetFlow
9 Open_vSwitch
10 CT_Zone
11 QoS
12 Datapath
13 SSL
14 Port
15 sFlow
16 Flow_Sample_Collector_Set
17 CT_Timeout_Policy
18 Mirror
19 Flow_Table
20 Interface
21 AutoAttach
22 Manager

```

Podemos verificar algumas tabelas e avaliar o que temos configurado até o momento, veja abaixo as colunas que temos na tabela Bridge:

```

1 # ovsdb-client -f list list-columns Bridge
2 Column      : name
3 Type        : "string"
4
5 Column      : flood_vlans
6 Type        : {"key":{"maxInteger":4095,"minInteger":0,"type":"integer"},"max":4096,"min":0}
7
8 Column      : stp_enable
9 Type        : "boolean"
10
11 Column      : ports
12 Type        : {"key":{"refTable":"Port","type":"uuid"},"max":"unlimited","min":0}
13
14 Column      : auto_attach
15 Type        : {"key":{"refTable":"AutoAttach","type":"uuid"},"min":0}
16
17 Column      : rstp_enable
18 Type        : "boolean"
19
20 Column      : _uuid
21 Type        : "uuid"
22
23 Column      : fail_mode
24 Type        : {"key":{"enum":["set",["secure","standalone"]],"type":"string"},"min":0}
25
26 Column      : rstp_status
27 Type        : {"key":"string","max":"unlimited","min":0,"value":"string"}
28
29 Column      : flow_tables
30 Type        : {"key":{"maxInteger":254,"minInteger":0,"type":"integer"},"max":"unlimited","min":0,"value"}
31
32 Column      : _version
33 Type        : "uuid"
34
35 Column      : netflow
36 Type        : {"key":{"refTable":"NetFlow","type":"uuid"},"min":0}
37
38 Column      : datapath_type
39 Type        : "string"
40
41 Column      : controller

```

```

42 Type                : {"key":{"refTable":"Controller","type":"uuid"},"max":"unlimited","min":0}
43
44 Column              : ipfix
45 Type                : {"key":{"refTable":"IPFIX","type":"uuid"},"min":0}
46
47 Column              : status
48 Type                : {"key":"string","max":"unlimited","min":0,"value":"string"}
49
50 Column              : other_config
51 Type                : {"key":"string","max":"unlimited","min":0,"value":"string"}
52
53 Column              : external_ids
54 Type                : {"key":"string","max":"unlimited","min":0,"value":"string"}
55
56 Column              : mirrors
57 Type                : {"key":{"refTable":"Mirror","type":"uuid"},"max":"unlimited","min":0}
58
59 Column              : datapath_id
60 Type                : {"key":"string","min":0}
61
62 Column              : mcast_snooping_enable
63 Type                : "boolean"
64
65 Column              : protocols
66 Type                : {"key":{"enum":["set",["OpenFlow10","OpenFlow11","OpenFlow12","OpenFlow13","OpenFlow14","0
67
68 Column              : sflow
69 Type                : {"key":{"refTable":"sFlow","type":"uuid"},"min":0}
70
71 Column              : datapath_version
72 Type                : "string"
73

```

Para analisar os dados que estão inseridos na tabela bridge, execute o comando abaixo:

```

1 # ovs-vsctl list bridge
2 _uuid                : 23f9c8c6-5c54-47a5-8dd2-cf23d8bca1c9
3 auto_attach          : []
4 controller           : []
5 datapath_id          : "0000c6c8f923a547"
6 datapath_type        : ""
7 datapath_version     : "<unknown>"
8 external_ids         : {}
9 fail_mode            : []
10 flood_vlans          : []
11 flow_tables          : {}
12 ipfix                : []
13 mcast_snooping_enable: false
14 mirrors              : []
15 name                 : mybridge
16 netflow              : []
17 other_config         : {}
18 ports                : [0ac9bcb5-6032-4b7c-98e5-b0b14fbbb79f, 97268cab-641a-4032-bf5a-287765a38f2a, d9e9caf8-3cf2
19 protocols            : []
20 rstp_enable          : false
21 rstp_status          : {}
22 sflow                : []

```

```

23 status          : {}
24 stp_enable       : false

```

Tabela Port:

```

1 # ovs-vsctl list port
2 _uuid          : d9e9caf8-3cf2-4788-86f4-09e8eb563334
3 bond_active_slave : []
4 bond_downdelay   : 0
5 bond_fake_iface  : false
6 bond_mode        : []
7 bond_updelay     : 0
8 cvlans           : []
9 external_ids     : {}
10 fake_bridge      : false
11 interfaces       : [e4608450-6d9e-47f4-b5a9-d10201828d8f]
12 lacp             : []
13 mac              : []
14 name             : mybridge
15 other_config     : {}
16 protected        : false
17 qos              : []
18 rstp_statistics  : {}
19 rstp_status      : {}
20 statistics       : {}
21 status           : {}
22 tag              : []
23 trunks           : []
24 vlan_mode        : []
25
26 _uuid          : 0ac9bcb5-6032-4b7c-98e5-b0b14fbbb79f
27 bond_active_slave : []
28 bond_downdelay   : 0
29 bond_fake_iface  : false
30 bond_mode        : []
31 bond_updelay     : 0
32 cvlans           : []
33 external_ids     : {}
34 fake_bridge      : false
35 interfaces       : [cd524274-e50e-4718-b79e-8fbdec2ca113]
36 lacp             : []
37 mac              : []
38 name             : vm1-if1
39 other_config     : {}
40 protected        : false
41 qos              : []
42 rstp_statistics  : {}
43 rstp_status      : {}
44 statistics       : {}
45 status           : {}
46 tag              : []
47 trunks           : []
48 vlan_mode        : []
49
50 _uuid          : 97268cab-641a-4032-bf5a-287765a38f2a
51 bond_active_slave : []
52 bond_downdelay   : 0
53 bond_fake_iface  : false
54 bond_mode        : []

```

```

55 bond_updelay      : 0
56 cvlans            : []
57 external_ids       : {}
58 fake_bridge        : false
59 interfaces         : [219f44f8-7bb6-4797-bd81-1605a583c094]
60 lacp               : []
61 mac               : []
62 name              : vm2-if1
63 other_config       : {}
64 protected          : false
65 qos               : []
66 rstp_statistics    : {}
67 rstp_status        : {}
68 statistics         : {}
69 status            : {}
70 tag               : []
71 trunks            : []
72 vlan_mode          : []
73

```

Tabela Interface:

```

1  # ovs-vsctl list interface
2  _uuid              : cd524274-e50e-4718-b79e-8fbdec2ca113
3  admin_state        : up
4  bfd               : {}
5  bfd_status         : {}
6  cfm_fault          : []
7  cfm_fault_status   : []
8  cfm_flap_count     : []
9  cfm_health         : []
10 cfm_mpid           : []
11 cfm_remote_mpid    : []
12 cfm_remote_opstate : []
13 duplex             : []
14 error              : []
15 external_ids       : {}
16 ifindex            : 0
17 ingress_policing_burst: 0
18 ingress_policing_kpkts_burst: 0
19 ingress_policing_kpkts_rate: 0
20 ingress_policing_rate: 0
21 lacp_current       : []
22 link_resets        : 1
23 link_speed         : []
24 link_state         : up
25 lldp              : {}
26 mac               : []
27 mac_in_use         : "7a:2e:15:c2:93:37"
28 mtu               : 1500
29 mtu_request        : []
30 name              : vm1-if1
31 ofport            : 1
32 ofport_request     : []
33 options            : {}
34 other_config       : {}
35 statistics         : {collisions=0, rx_bytes=2868, rx_crc_err=0, rx_dropped=2, rx_errors=0, rx_frame_err=0, rx
36 status            : {driver_name=openvswitch}
37 type              : internal

```

```
38
39 _uuid          : 219f44f8-7bb6-4797-bd81-1605a583c094
40 admin_state    : up
41 bfd            : {}
42 bfd_status     : {}
43 cfm_fault      : []
44 cfm_fault_status : []
45 cfm_flap_count : []
46 cfm_health     : []
47 cfm_mpid       : []
48 cfm_remote_mpid : []
49 cfm_remote_opstate : []
50 duplex         : []
51 error          : []
52 external_ids   : {}
53 ifindex        : 0
54 ingress_policing_burst: 0
55 ingress_policing_kpkts_burst: 0
56 ingress_policing_kpkts_rate: 0
57 ingress_policing_rate: 0
58 lacp_current   : []
59 link_resets     : 1
60 link_speed     : []
61 link_state     : up
62 lldp           : {}
63 mac            : []
64 mac_in_use     : "b6:93:74:80:62:23"
65 mtu            : 1500
66 mtu_request    : []
67 name           : vm2-if1
68 ofport         : 2
69 ofport_request : []
70 options        : {}
71 other_config    : {}
72 statistics     : {collisions=0, rx_bytes=2380, rx_crc_err=0, rx_dropped=8, rx_errors=0, rx_frame_err=0, rx
73 status         : {driver_name=openvswitch}
74 type           : internal
75
76 _uuid          : e4608450-6d9e-47f4-b5a9-d10201828d8f
77 admin_state    : up
78 bfd            : {}
79 bfd_status     : {}
80 cfm_fault      : []
81 cfm_fault_status : []
82 cfm_flap_count : []
83 cfm_health     : []
84 cfm_mpid       : []
85 cfm_remote_mpid : []
86 cfm_remote_opstate : []
87 duplex         : []
88 error          : []
89 external_ids   : {}
90 ifindex        : 5
91 ingress_policing_burst: 0
92 ingress_policing_kpkts_burst: 0
93 ingress_policing_kpkts_rate: 0
94 ingress_policing_rate: 0
95 lacp_current   : []
```

```

96 link_resets      : 1
97 link_speed       : []
98 link_state       : up
99 lldp             : {}
100 mac              : []
101 mac_in_use       : "c6:c8:f9:23:a5:47"
102 mtu              : 1500
103 mtu_request      : []
104 name             : mybridge
105 ofport           : 65534
106 ofport_request   : []
107 options          : {}
108 other_config     : {}
109 statistics       : {collisions=0, rx_bytes=1956, rx_crc_err=0, rx_dropped=0, rx_errors=0, rx_frame_err=0, rx
110 status           : {driver_name=openvswitch}
111 type             : internal

```

É interessante também avaliar os flows que o ovs-vswitchd tem programado para instalar caso o Datapath requisite:

```

1 # ovs-ofctl dump-flows mybridge
2 cookie=0x0, duration=8791.539s, table=0, n_packets=95, n_bytes=7666, priority=0 actions=NORMAL

```

Para analisarmos o **Datapath** e os flows existentes no fast path, use o comando abaixo porém gere algum tipo de tráfego entre os namespaces (icmp) para verificar os flows instalados:

```

1 # ip netns exec ns2_vm2 ping 192.168.1.10
2 PING 192.168.1.10 (192.168.1.10) 56(84) bytes of data.
3 64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=0.208 ms
4 64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.066 ms
5 ^C
6 --- 192.168.1.10 ping statistics ---
7 2 packets transmitted, 2 received, 0% packet loss, time 1027ms
8 rtt min/avg/max/mdev = 0.066/0.137/0.208/0.071 ms
9 # ovs-dpctl dump-flows
10 recirc_id(0),in_port(3),eth(src=6e:de:5e:2f:19:02,dst=0a:10:30:ea:2c:4a),eth_type(0x0800),ipv4(frag=no), packets
11 recirc_id(0),in_port(2),eth(src=0a:10:30:ea:2c:4a,dst=6e:de:5e:2f:19:02),eth_type(0x0800),ipv4(frag=no), packets
12

```

Conclusão

Neste treinamento foi possível entender melhor sobre o Open vSwitch (OVS) na prática, avançar um pouco na arquitetura de seus componentes e os tools envolvidos.

Abaixo existe o passo a passo para realizar o cleanup na VM utilizada para os exercícios.

```

1 # apt remove -y openvswitch-switch openvswitch-common python3-openvswitch
2 # rm -rf /var/lib/openvswitch
3 # rm -rf /var/log/openvswitch
4 # rm -rf /etc/openvswitch
5 # rm -f /etc/default/openvswitch-switch
6 # ip netns delete ns1_vm1
7 # ip netns delete ns2_vm2
8 # reboot

```